

Mathematical Description of a Two-Layer Neural Network for Digit Classification

1 Problem setting and notation

We consider multiclass classification with $K = 10$ classes (digits 0–9). Each input image is a vector $\mathbf{x} \in \mathbb{R}^d$ with $d = 784$ (a 28×28 image flattened). The dataset is

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m, \quad y^{(i)} \in \{0, 1, \dots, K-1\}.$$

The implementation normalizes pixel values by 255, so we assume $\mathbf{x}$ has entries in $[0, 1]$. For minibatch-free training in the code, the full training set is used per iteration.

Matrix convention (matching the code). Inputs are stacked by columns:

$$X = [\mathbf{x}^{(1)} \quad \dots \quad \mathbf{x}^{(m)}] \in \mathbb{R}^{d \times m}.$$

Labels are stored as $\mathbf{y} = (y^{(1)}, \dots, y^{(m)})$. The one-hot encoding is

$$Y \in \mathbb{R}^{K \times m}, \quad Y_{k,i} = \begin{cases} 1, & k = y^{(i)}, \\ 0, & \text{otherwise.} \end{cases}$$

2 Model architecture

The network is a fully-connected (dense) two-layer model with one hidden layer of width $h = 64$:

$$d = 784 \longrightarrow h = 64 \longrightarrow K = 10.$$

Parameters:

$$W_1 \in \mathbb{R}^{h \times d}, \quad b_1 \in \mathbb{R}^{h \times 1}, \quad W_2 \in \mathbb{R}^{K \times h}, \quad b_2 \in \mathbb{R}^{K \times 1}.$$

3 Forward propagation

Given $X \in \mathbb{R}^{d \times m}$, the forward pass computes

$$\begin{aligned} Z_1 &= W_1 X + b_1, & Z_1 &\in \mathbb{R}^{h \times m}, \\ A_1 &= \text{ReLU}(Z_1), & A_1 &\in \mathbb{R}^{h \times m}, \\ Z_2 &= W_2 A_1 + b_2, & Z_2 &\in \mathbb{R}^{K \times m}, \\ A_2 &= \text{softmax}(Z_2), & A_2 &\in \mathbb{R}^{K \times m}. \end{aligned}$$

3.1 ReLU nonlinearity

The rectified linear unit is applied elementwise:

$$\text{ReLU}(z) = \max\{0, z\}.$$

Its derivative (subgradient at 0 ignored, as in typical implementations) is

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

3.2 Softmax and predicted probabilities

For a single example with logits $\mathbf{z} \in \mathbb{R}^K$, softmax gives class probabilities

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

In matrix form (applied columnwise), A_2 contains probabilities and satisfies $\sum_{k=1}^K (A_2)_{k,i} = 1$ for each i .

To improve numerical stability, the code uses the standard shift $\mathbf{z} \mapsto \mathbf{z} - \max_k z_k$, which leaves softmax unchanged.

4 Loss function

The code's backpropagation corresponds to the (average) softmax cross-entropy loss

$$\mathcal{L}(W_1, b_1, W_2, b_2) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K Y_{k,i} \log(A_2)_{k,i}.$$

Equivalently, for each example i with true class $y^{(i)}$, $\ell^{(i)} = -\log(A_2)_{y^{(i)},i}$ and $\mathcal{L} = \frac{1}{m} \sum_i \ell^{(i)}$.

5 Backpropagation (gradients)

We compute gradients of $\mathcal{L}$ with respect to all parameters. Let m denote the number of training examples.

5.1 Output layer gradients

A standard result for softmax + cross-entropy gives

$$\frac{\partial \mathcal{L}}{\partial Z_2} = A_2 - Y,$$

so define

$$dZ_2 := A_2 - Y \in \mathbb{R}^{K \times m}.$$

Then

$$\begin{aligned} dW_2 &:= \frac{\partial \mathcal{L}}{\partial W_2} = \frac{1}{m} dZ_2 A_1^\top \in \mathbb{R}^{K \times h}, \\ db_2 &:= \frac{\partial \mathcal{L}}{\partial b_2} = \frac{1}{m} \sum_{i=1}^m dZ_2^{(:,i)} \in \mathbb{R}^{K \times 1}, \end{aligned}$$

where $dZ_2^{(:,i)}$ denotes the i th column.

5.2 Hidden layer gradients

Propagating through the affine map and ReLU:

$$\begin{aligned} dA_1 &= W_2^\top dZ_2 \in \mathbb{R}^{h \times m}, \\ dZ_1 &= dA_1 \odot \text{ReLU}'(Z_1) = (W_2^\top dZ_2) \odot \mathbf{1}_{\{Z_1 > 0\}} \in \mathbb{R}^{h \times m}, \end{aligned}$$

where $\odot$ is the Hadamard (elementwise) product and $\mathbf{1}_{\{Z_1 > 0\}}$ is the elementwise indicator.

Then

$$\begin{aligned} dW_1 &:= \frac{\partial \mathcal{L}}{\partial W_1} = \frac{1}{m} dZ_1 X^\top \in \mathbb{R}^{h \times d}, \\ db_1 &:= \frac{\partial \mathcal{L}}{\partial b_1} = \frac{1}{m} \sum_{i=1}^m dZ_1^{(:,i)} \in \mathbb{R}^{h \times 1}. \end{aligned}$$

6 Parameter update (gradient descent)

With learning rate $\alpha > 0$, a full-batch gradient descent step is

$$\begin{aligned} W_1 &\leftarrow W_1 - \alpha dW_1, & b_1 &\leftarrow b_1 - \alpha db_1, \\ W_2 &\leftarrow W_2 - \alpha dW_2, & b_2 &\leftarrow b_2 - \alpha db_2. \end{aligned}$$

Repeating this for a fixed number of iterations yields the trained parameters.

7 Initialization

The weights are initialized with a He/Kaiming-type scaling appropriate for ReLU:

$$(W_1)_{ij} \sim \mathcal{N}\left(0, \frac{2}{d}\right), \quad (W_2)_{ij} \sim \mathcal{N}\left(0, \frac{2}{h}\right),$$

and biases are initialized to 0. This aims to keep activation variances roughly controlled across layers.

8 Prediction and accuracy

Given A_2 , the predicted class for example i is

$$\hat{y}^{(i)} = \arg \max_{k \in \{0, \dots, K-1\}} (A_2)_{k,i}.$$

Accuracy on a dataset is

$$\text{Acc} = \frac{1}{m} \sum_{i=1}^m \mathbf{1}\{\hat{y}^{(i)} = y^{(i)}\}.$$